

Sera FX Onchain

Security Assessment

CertiK Assessed on Apr 30th, 2026





CertiK Assessed on Apr 30th, 2026

Sera FX Onchain

The security assessment was prepared by CertiK.

Executive SummaryTYPES
ERC-20ECOSYSTEM
EVM CompatibleMETHODS
Manual Review, Static AnalysisLANGUAGE
SolidityTIMELINE
Preliminary comments published on 03/24/2026
Final report published on 04/30/2026**Vulnerability Summary**

17

Total Findings

11

Resolved

1

Multi-Sig & Timelock

0

Partially Resolved

5

Acknowledged

0

Declined

1 Centralization

1 Multi-Sig & Timelock

Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.

0 Critical

Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.

0 Major

Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.

3 Medium

3 Resolved

Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.

8 Minor

7 Resolved, 1 Acknowledged

Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.

5 Informational

1 Resolved, 4 Acknowledged

Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | SERA FX ONCHAIN

Audit Summary

[Executive Summary](#)

[Vulnerability Summary](#)

[Codebase](#)

[Audit Scope](#)

[Approach & Methods](#)

Review Notes

[Overview](#)

[Sera](#)

[Vault](#)

[SeraSOR](#)

[SeraBatcher](#)

[External Dependencies](#)

[Privileged Functions](#)

Findings

[SFO-03 : Centralization Related Risks](#)

[SFO-04 : Incorrect EIP-2098 Signature Decoding Breaks Permit Deposits](#)

[SFO-12 : Positive Slippage Misallocated By Order Position Instead Of Maker/Taker Role](#)

[SFO-13 : Unbound Partial Execution Can Sweep Taker Funds To Treasury](#)

[SFO-05 : Same-Token Matching Weakens Withdrawal Control Boundaries](#)

[SFO-06 : Identical Order Self-Match Corrupts Fill Accounting](#)

[SFO-08 : Missing Zero Address Validation](#)

[SFO-11 : Non-Final Routed Legs Do Not Enforce `order0.recipient == address\(sera\)`](#)

[SFO-15 : Unnormalized `v` In 65-Byte Permit Signatures May Break EIP-2612 Integration](#)

[SFO-16 : `maxInputAmount` Is Enforced On Gross Staged Input Rather Than Net Taker Spend](#)

[SFO-17 : Intent Replay Semantics Depend On Single-Versus-Multi Router Deployments](#)

[SFO-18 : Delayed Withdrawal Not Bound To Balance Weakens Cooldown Mechanism](#)

[SFO-01 : Permit Function Doesn't Comply With DAI Token](#)

[SFO-07 : Partially Filled Orders Ignore Later Whitelist/Min-Amount Policy Updates](#)

[SFO-09 : Inconsistent Blacklist Enforcement](#)

[SFO-14 : Order Lifetime Tied To Execution Time Instead Of Signing Time](#)

SFO-20 : Partial-Fill Rounding Behavior Under Fragmented Execution

I Optimizations

SFO-02 : State variables that could be declared immutable

I Appendix

I Disclaimer

CODEBASE | SERA FX ONCHAIN

Repository

<https://github.com/sera-cx/orderbook-contract-v2>

Commit

Initial:

- [738f0a8579d7ce0184cc5ce1e4406083b103b063](#)

Update:

- [573a50dced379594ab5dbca5cc10a86802b716da](#)
- [905e7f7ba3a669fa0b2e393afeb289e5189c04f6](#)
- [c56510f76c5935f5f912058ab910e015e3af0f40](#)

Final:

- [dd974081b50c4cb378ca5312611ef5574c56b4d9](#)

Final file hashes (SHA-256):

- **Sera.sol:** 3b9c1043acd4b9e96c89a03c01745ea72380d4f5b1a7fc835f05b396339993dc
- **SeraAdmin.sol:** 03c5835b839b58ead8de9acceae8d96e6c73b954f8ba8b7612b31cf1bd638f78
- **SeraBase.sol:** abb3f9aef8edba615ac7b62b90ff226fede8066cc9d5960b50bf00e86ed3f347
- **SeraBatcher.sol:** 3d35475044189dada90c89617962900c32d258dcfd0b61c621c035e6e330c6af
- **SeraLib.sol:** 208294b2af9093841cb7987cda59d461877ca8a05fde8b6980e5ad87e08c094a
- **SeraSOR.sol:** 1c5dc8d04ce53c630b0a45c68b2c76b195ac4d5b59c47ba550ce54cbf1aa2b9f
- **Vault.sol:** 5bb787c96fc654a75042093ac4a6a6763eceaabcf6e6a5261b4e9ae2321ad99f

Audit Scope

The file in scope is listed in the appendix.

APPROACH & METHODS | SERA FX ONCHAIN

This audit was conducted for Sera to evaluate the security and correctness of the smart contracts associated with the Sera FX Onchain project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of Manual Review and Static Analysis.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Dynamic testing to validate runtime behavior and assess execution risks.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

REVIEW NOTES | SERA FX ONCHAIN

Overview

The **Sera** protocol is designed as a hybrid execution model that combines off-chain order coordination with on-chain cryptographic settlement. The architecture separates custody (`Vault`), core matching (`Sera`), route execution (`SeraSOR`), and batching (`SeraBatcher`) into distinct modules. Users sign EIP-712 payloads for orders, routes, and withdrawal intents, while authorized executors submit settlement transactions on-chain.

Sera

The `Sera` contract is the primary on-chain matching and settlement engine. It validates signed order data, enforces expiry and fill constraints, and records incremental fill state via `filledAmount`. Settlement supports both internal ledger transfers and direct token payouts depending on recipient configuration. The contract also includes two withdrawal paths: a delayed user-triggered emergency flow and a dual-signature instant withdrawal flow. Inherited admin state controls treasury routing, token allowlisting, minimum amounts, and configurable slippage-share parameters used during settlement.

It should be noted that the protocol and maker spread shares are floor-rounded, while the taker receives the remaining portion. This creates a small, deterministic rounding bias in favor of the taker. According to the project documentation, which is intended behavior.

Vault

The `Vault` contract serves as the custody and accounting layer for user assets. It maintains per-token user balances and tracked aggregate balances, and exposes trader-gated functions for deposit, withdrawal, internal ledger transfer, and ledger crediting. This ledger-first design enables net settlement without forcing token transfers for every matched leg. The module also contains administrative controls for blacklist management and surplus-token rescue operations.

SeraSOR

The `SeraSOR` contract implements atomic multi-leg route execution. It binds route integrity through `routeHash`, verifies a single taker route signature, and settles each leg through `Sera.settleRoutedLeg`. For routing efficiency, intermediate outputs may be held transiently and reused across subsequent legs instead of being re-deposited each hop. At route completion, residual transient balances are swept via the protocol settlement path to finalize accounting.

SeraBatcher

The `SeraBatcher` contract provides batch execution wrappers around the core `Sera` matching primitive. It supports continue-on-error batches, atomic all-or-nothing batches, and mixed execution combining atomic sub-batches with independent matches. The module emits structured failure metadata and bitmask results to support deterministic off-chain reconciliation.

External Dependencies

The **Sera** protocol relies on a small set of third-party libraries and standards to implement role management, signature

verification, token transfer safety, and arithmetic.

The following are external dependency contracts and libraries used within the audited codebase:

- @openzeppelin/contracts/access/AccessControl.sol
- @openzeppelin/contracts/token/ERC20/IERC20.sol
- @openzeppelin/contracts/token/ERC20/extensions/IERC20Permit.sol
- @openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol
- @openzeppelin/contracts/utils/Pausable.sol
- @openzeppelin/contracts/utils/ReentrancyGuardTransient.sol
- @openzeppelin/contracts/utils/math/Math.sol
- solady/src/utis/EIP712.sol
- solady/src/utis/ECDSA.sol

It is assumed that these dependencies are trusted and behave according to their published specifications.

Additionally, token contracts integrated by the protocol are treated as external dependencies, and their runtime behavior (including `transfer`, `transferFrom`, and `permit` semantics) is assumed to be compatible with the protocol's custody and settlement flow. In line with the project documentation, this integration model is intended for standard ERC-20 tokens and does not account for fee-on-transfer token mechanics.

Privileged Functions

In the **Sera** protocol, some privileged roles are adopted to ensure the dynamic runtime updates of the project, which are specified in the **Centralization Related Risks** finding.

The advantage of those privileged roles in the codebase is that the client reserves the ability to adjust the protocol according to the runtime required to best serve the community.

It is also worth noting the potential drawbacks of these functions, which should be clearly stated through the client's action/plan. Additionally, if the private keys of the privileged accounts are compromised, it could lead to devastating consequences for the project.

To improve the trustworthiness of the project, dynamic runtime updates in the project should be notified to the community. Currently, the project leverages a multisig wallet and a Timelock contract to mitigate centralization risks. The detailed configuration can be found in the **Centralization Related Risks** finding.

FINDINGS | SERA FX ONCHAIN



17
Total Findings

0
Critical

1
Centralization

0
Major

3
Medium

8
Minor

5
Informational

This report has been prepared for Sera to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 17 issues were identified. Leveraging a combination of Manual Review & Static Analysis the following findings were uncovered:

ID	Title	Category	Severity	Status
SFO-03	Centralization Related Risks	Centralization	Centralization	● 4/5 Multi-Sig, 48h Timelock
SFO-04	Incorrect EIP-2098 Signature Decoding Breaks Permit Deposits	Logical Issue	Medium	● Resolved
SFO-12	Positive Slippage Misallocated By Order Position Instead Of Maker/Taker Role	Logical Issue	Medium	● Resolved
SFO-13	Unbound Partial Execution Can Sweep Taker Funds To Treasury	Logical Issue	Medium	● Resolved
SFO-05	Same-Token Matching Weakens Withdrawal Control Boundaries	Logical Issue	Minor	● Resolved
SFO-06	Identical Order Self-Match Corrupts Fill Accounting	Logical Issue	Minor	● Resolved
SFO-08	Missing Zero Address Validation	Volatile Code	Minor	● Resolved
SFO-11	Non-Final Routed Legs Do Not Enforce <code>order0.recipient == address(sera)</code>	Design Issue	Minor	● Resolved

ID	Title	Category	Severity	Status
SFO-15	Unnormalized <code>v</code> In 65-Byte Permit Signatures May Break EIP-2612 Integration	Logical Issue	Minor	● Resolved
SFO-16	<code>maxInputAmount</code> Is Enforced On Gross Staged Input Rather Than Net Taker Spend	Logical Issue	Minor	● Acknowledged
SFO-17	Intent Replay Semantics Depend On Single-Versus-Multi Router Deployments	Design Issue	Minor	● Resolved
SFO-18	Delayed Withdrawal Not Bound To Balance Weakens Cooldown Mechanism	Logical Issue	Minor	● Resolved
SFO-01	Permit Function Doesn't Comply With DAI Token	Design Issue	Informational	● Acknowledged
SFO-07	Partially Filled Orders Ignore Later Whitelist/Min-Amount Policy Updates	Logical Issue	Informational	● Acknowledged
SFO-09	Inconsistent Blacklist Enforcement	Inconsistency, Logical Issue	Informational	● Acknowledged
SFO-14	Order Lifetime Tied To Execution Time Instead Of Signing Time	Design Issue	Informational	● Acknowledged
SFO-20	Partial-Fill Rounding Behavior Under Fragmented Execution	Design Issue	Informational	● Resolved

SFO-03 | Centralization Related Risks

Category	Severity	Location	Status
Centralization	● Centralization		● 4/5 Multi-Sig, 48h Timelock

Description

In the contract `Sera`, the role `DEFAULT_ADMIN_ROLE` has authority over the following functions:

- `setTrustedRouter()`
- `grantRole()`
- `revokeRole()`

Any compromise to the `DEFAULT_ADMIN_ROLE` account may allow an attacker to reconfigure trusted execution paths and arbitrarily grant or revoke privileged roles, which can alter protocol control boundaries.

In the contract `Sera`, the role `EXECUTOR_ROLE` has authority over the following functions:

- `matchOrders()`

Any compromise to the `EXECUTOR_ROLE` account may allow an attacker to control when and how order matches are executed.

In the contract `Sera`, the role `EXECUTOR_ROLE_CACHED` has authority over the following functions:

- `sweepTransientToProtocol()`

Any compromise to the `EXECUTOR_ROLE_CACHED` account may allow an attacker to trigger treasury-directed sweeping of transient balances during routed execution flows.

In the contract `SeraAdmin`, the role `DEFAULT_ADMIN_ROLE` has authority over the following functions:

- `setTreasury()`
- `setSlippageShares()`
- `batchModifyWhitelistedTokens()`
- `rescueToken()`
- `grantRole()`
- `revokeRole()`

Any compromise to the `DEFAULT_ADMIN_ROLE` account may allow an attacker to change treasury routing, reconfigure fee/slippage parameters, modify token allowlists and minimum thresholds, rescue tokens from the contract, and reassign privileged roles.

In the contract `SeraAdmin`, the role `PAUSER_ROLE` has authority over the following functions:

- `pause()`
- `unpause()`

Any compromise to the `PAUSER_ROLE` account may allow an attacker to arbitrarily halt or resume protocol operations gated by pause checks.

In the contract `SeraSOR`, the role `EXECUTOR_ROLE_CACHED` has authority over the following functions:

- `executeRoute()`

Any compromise to the `EXECUTOR_ROLE_CACHED` account may allow an attacker to control the submission and execution timing of routed multi-leg settlements.

In the contract `SeraBatcher`, the role `EXECUTOR_ROLE_CACHED` has authority over the following functions:

- `batchMatchOrders()`
- `batchMatchOrdersAtomic()`
- `batchMatchMixed()`

Any compromise to the `EXECUTOR_ROLE_CACHED` account may allow an attacker to control batch execution semantics and the sequencing of large sets of order matches.

In the contract `Vault`, the role `TRADER_ROLE` has authority over the following functions:

- `deposit()`
- `creditLedger()`
- `withdraw()`
- `transferLedger()`

Any compromise to the `TRADER_ROLE` account may allow an attacker to move user balances within the custody layer, trigger withdrawals to arbitrary recipients, and update internal accounting balances.

In the contract `Vault`, the role `DEFAULT_ADMIN_ROLE` has authority over the following functions:

- `setBlacklisted()`
- `rescueToken()`
- `grantRole()`
- `revokeRole()`

Any compromise to the `DEFAULT_ADMIN_ROLE` account may allow an attacker to alter user blacklist status, extract surplus assets, and arbitrarily reassign operational roles.

Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend

centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets.

Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign ($\frac{2}{3}$, $\frac{3}{5}$) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;\

AND

- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;\

AND

- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;\

AND

- Introduction of a DAO/governance/voting module to increase transparency and user involvement.\

AND

- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.\

OR

- Remove the risky functionality.

I Alleviation

[CertiK, 04/23/2026]:

The team has deployed the contracts based on the specified commit [dd974081b50c4cb378ca5312611ef5574c56b4d9](#), with the corresponding contract addresses listed below:

- **Vault:** [0xabf1f322c6626f283895597be3b8d52fc618ec21](#)
- **Sera:** [0xf25bf48392e9e402ef2661bf7530d0420716b565](#)
- **SeraSOR:** [0x5a999e11a6959dbaafab93d2f64cd82497891681](#)
- **SeraBatcher:** [0x238399910636f41f351d20411ffae4deb8f2b9a5](#)

Under the current configuration, the team has introduced a **4/5** multisig wallet and a **Timelock** contract to mitigate centralization risks. The **Timelock** contract has been granted the `DEFAULT_ADMIN_ROLE` for both the `Vault` and `Sera` contracts.

- **Transaction hashes:**
 - [0x23ed53e566d933c463bcd0191fc8f0240665071ba8051a367b29756458a0d02d](#)
 - [0x25d7b42a8fadc54dbc22c7ade604c83b690c6cf65303ddabddf246554f52601c](#)
- **Timelock:** [0x7dD3AB088e5b6F4cd0c366eDaE81C5AC10A6e779](#)

In addition, the multisig wallet is set as the admin of the **Timelock** contract.

- **Transaction hash:** [0x9083c3083b2ac9bc590faccace0cc2e69abfc24c2b4c9839b103a66cf879be4c](#)

The multisig configuration is as follows:

- **Safe multisig wallet:** [0x562ffc562d3d70e04b3160cd80d612dad5c0136](#)
- **Signers:**
 - 0xa88cA1Ea4F78e11ba5DDCeAFc8baF7ff5Be4cb7e
 - 0x9b6367da16Ed37706276a9AadD9406772d6da949
 - 0xdB98Ae259913F05D184bC2e11f84F51180e5D6D6
 - 0x38be932f24d8B8B04dfd07DbcAE8ee180C68D99E
 - 0x0901977ad401821c46dA98E8Aeb33cB704fEC97A

SFO-04 | Incorrect EIP-2098 Signature Decoding Breaks Permit Deposits

Category	Severity	Location	Status
Logical Issue	Medium	src/Sera.sol (03/23-738f0a8): 400~412	Resolved

Description

The `depositFundWithPermit(...)` function is intended to support both 65-byte standard signatures and 64-byte EIP-2098 compact signatures, but the current decoding logic only matches the 65-byte `(r, s, v)` layout.

Although the function explicitly accepts `_sig.length == 64`, it still parses calldata as if a separate `v` byte exists:

- `r` is read from the first 32 bytes (correct for both formats),
- the second 32-byte word is assigned directly to `s` (incorrect for compact signatures, where this word is `vs`),
- `v` is read from a non-existent 65th byte and then normalized to `27`.

```
function depositFundWithPermit(address _token, address _owner, uint256
_permitAmount, uint256 _depositAmount, uint256 _deadline, bytes calldata _sig)
external whenNotPaused {
    if (!tokenConfigs[_token].isWhitelisted) revert TokenNotWhitelisted(_token);
    if (_depositAmount > _permitAmount) revert AmountMismatch();
    // Check if allowance is already sufficient (e.g., if front-run or
    previously permitted)
    if (IERC20(_token).allowance(_owner, address(vault)) < _depositAmount) {
        if (_sig.length != 65 && _sig.length != 64) revert
InvalidSignatureLength();
        bytes32 r;
        bytes32 s;
        uint8 v;
        assembly {
            r := calldataload(_sig.offset)
            s := calldataload(add(_sig.offset, 0x20))
            v := byte(0, calldataload(add(_sig.offset, 0x40)))
        }
        if (v < 27) v += 27;
        try IERC20Permit(_token).permit(_owner, address(vault), _permitAmount,
_deadline, v, r, s) {} catch {}
    }
    vault.deposit(_owner, _token, _depositAmount);
}
```

For a valid 64-byte EIP-2098 signature, this causes two decoding errors:

1. `vs` is used as `s` without clearing the highest bit (which in EIP-2098 encodes `v` parity), so the derived `s` may violate low-`s` constraints.

2. `v` is reconstructed incorrectly (typically forced to `27`), which can mismatch the original signature parity (especially when the real parity corresponds to `v = 28`).

The malformed `(v, r, s)` tuple is then passed to `permit(...)`, which may revert as invalid/malleable. Because this call is wrapped in `try/catch {}`, the revert is silently swallowed, and execution continues to `vault.deposit(...)`, which later fails due to missing allowance.

As a result, compact-signature permit deposits are unreliable and operationally confusing: the user sees an allowance/transfer failure path rather than a clear signature-decoding failure.

Recommendation

Consider implementing signature decoding by format:

- If length is `65`, decode as `(r, s, v)`.
- If length is `64`, decode as EIP-2098 `(r, vs)`.

Additionally, avoid silently swallowing permit failures; emit/revert with a clear permit-specific error so signature issues are diagnosable instead of surfacing later as generic allowance failures.

Alleviation

[Sera, 03/28/2026]:

Issue acknowledged. Changes have been reflected in the commit hash: [7ee8302c8c821fabe35b7ce2b6f719aafde280e3](#).

[CertiK, 04/02/2026]:

The above issue has been resolved in this submission. However, the externally imported library contracts do not address the signature malleability issue.

Solady's `ECDSA.recover` variants decode `(r, s, v)` (including EIP-2098 compact 64-byte form) and call the `ecrecover` precompile, but they do not enforce a canonical (non-malleable) signature**. In particular, they do not reject “high-`s`” values (`s > secp256k1_n / 2`). For any valid ECDSA signature `(r, s, v)` on a fixed digest, another valid signature typically exists as `(r, n - s, v')` with flipped recovery id. Without a low-`s` (EIP-2 style) check or equivalent canonicalization, the same authorization can be represented by more than one byte string, which breaks any assumption that “the signature bytes are a unique identifier,” and can complicate replay or bookkeeping that keys off raw signature data.

SoladyECDSA:

```
/// WARNING! Do NOT directly use signatures as unique identifiers:
/// - The recovery operations do NOT check if a signature is non-malleable.
/// - Use a nonce in the digest to prevent replay attacks on the same contract.
/// - Use EIP-712 for the digest to prevent replay attacks across different chains
    and contracts.
///   EIP-712 also enables readable signing of typed data for better user safety.
/// - If you need a unique hash from a signature, please use the `canonicalHash`
    functions.
```


Consider using OpenZeppelin's `ECDSA` for recovery so signatures are verified with a low-`s` (EIP-2) check, which blocks common ECDSA malleability from alternate `(r, s, v)` encodings of the same authorization.

[Sera, 04/09/2026]:

Issue acknowledged. Changes have been reflected in the commit hash: [905e7f7ba3a669fa0b2e393afeb289e5189c04f6](#).

[CertiK, 04/10/2026]:

We have two questions regarding this point that we would like to confirm with the team:

1. User signatures are verified via `SignatureChecker.isValidSignatureNowCalldata()`. This means that if the signer is an EOA, standard 65-byte signature verification is used. If the signer is a contract account, the verification is delegated to the contract itself (e.g., contract wallets like Safe implementing ERC-1271), without being constrained by fixed signature lengths such as 64/65 bytes, and the verification logic is fully defined by the contract.
2. The signature verification mechanism for `EXECUTOR_ROLE` differs from that of user signatures. If the team intends to support contract wallets (e.g., Safe) for `EXECUTOR_ROLE` as well, the current approach may fail and may need to be aligned with the same verification method used for user signatures.

[Sera, 04/13/2026]:

We have added support for contract signing only, following the EIP-1271 standard.

[CertiK, 04/10/2026]:

We re-checked the implementation and found that EIP-1271 support is currently applied to user-side signatures (via `SignatureChecker.isValidSignatureNowCalldata()`), which is great.

However, in `executeInstantWithdrawDualSig`, the `executorSignature` path still uses `ECDSA.parseCalldata + ECDSA.recover` with a 64/65-byte length constraint. This remains EOA-style verification and does not provide ERC-1271-compatible executor signing.

So at this stage, the implementation appears to be: contract-signing support for users, but not yet for executor signatures in the dual-sig withdraw flow.

[Sera, 04/21/2026]:

The latest commit allows EIP-7702, EIP-1271, and EOA executor. Changes have been reflected in the commit hash: [c56510f76c5935f5f912058ab910e015e3af0f40](#).

[CertiK, 04/21/2026]:

The signature verification flow for `EXECUTOR_ROLE` is now aligned with that of user signatures. It is worth noting that EIP-1271 signature verification relies on the signer contract's external `isValidSignature` implementation for signature verification. As such, the correctness and security of the verification process are dependent on the signer's implementation, which is not assessed as part of this review.

SFO-12 Positive Slippage Misallocated By Order Position Instead Of Maker/Taker Role

Category	Severity	Location	Status
Logical Issue	 Medium	src/Sera.sol (03/23-738f0a8): 254~282; src/SeraSOR.sol (03/23-738f0a8): 78	 Resolved

Description

The `_calculateSettlement(...)` function computes positive-slippage components using maker/taker-configured share parameters but applies them through `executionValue0` / `executionValue1`, which are position/flow variables tied to `order0` and `order1` payment direction.

```

function _calculateSettlement(MatchData calldata _match, uint256
executionValue0, uint256 executionValue1, bytes32 orderHash0, bytes32 orderHash1)
internal returns (SettlementCalc memory calc) {
    SlippageShare memory shares = slippageShares;
    // Protocol takes protocol fee AND its configured portion of the spread
    calc.protocolFee0 = Math.mulDiv(executionValue1, _match.order1.feeBps,
BPS_DENOMINATOR);
    calc.protocolFee1 = Math.mulDiv(executionValue0, _match.order0.feeBps,
BPS_DENOMINATOR);
    uint256 totalSpread0 = _match.matchAmount0 - executionValue1;
    uint256 totalSpread1 = _match.matchAmount1 - executionValue0;
    uint256 protocolSpread0 = Math.mulDiv(totalSpread0, shares.protocolShareBps,
shares.totalBps);
    // Taker receives what's left
    @> uint256 takerBonus0 = totalSpread0 - protocolSpread0 -
Math.mulDiv(totalSpread0, shares.makerShareBps, shares.totalBps);
    uint256 protocolSpread1 = Math.mulDiv(totalSpread1, shares.protocolShareBps,
shares.totalBps);
    @> uint256 makerBonus1 = Math.mulDiv(totalSpread1, shares.makerShareBps,
shares.totalBps);
    // Refund non-protocol spread to users by manipulating payouts.
    // Token 0 surplus (USDT) bonuses:
    // - Taker explicitly receives their `takerBonus0` via executionValue1
payout increase.
    // - Maker implicitly receives `makerBonus0` as a rebate (by NOT pulling it
from them for executionValue1).
    @> calc.executionValue1 = executionValue1 + takerBonus0;
    // Token 1 surplus (SGD) bonuses:
    // - Maker explicitly receives their `makerBonus1` via executionValue0
payout increase.
    // - Taker implicitly receives `takerBonus1` as a rebate.
    @> calc.executionValue0 = executionValue0 + makerBonus1;
    // Protocol take is the sum of protocol fee and spread
    calc.protocolTake0 = calc.protocolFee0 + protocolSpread0;
    calc.protocolTake1 = calc.protocolFee1 + protocolSpread1;
    filledAmount[orderHash0] += _match.matchAmount0;
    filledAmount[orderHash1] += _match.matchAmount1;
    calc.order0FullyFilled = (filledAmount[orderHash0] >=
_match.order0.fromAmount);
    calc.order1FullyFilled = (filledAmount[orderHash1] >=
_match.order1.fromAmount);
}

```

As a result, settlement behavior is effectively position-based rather than role-based. This creates a role-mapping inconsistency: depending on how orders are oriented, user-side slippage rewards can be assigned to the opposite economic role.

In routed settlement, the mismatch is explicit because the `executeRoute(...)` function defines `order0` as taker and `order1` as maker, while the `_calculateSettlement(...)` function still applies bonus flows by `order0/order1` payout

channels. In standalone matching, where no canonical maker/taker orientation is enforced, payout outcomes can vary with order ordering when share parameters are asymmetric.

```
function executeRoute(MatchData[] calldata matches, bytes calldata
routeSignature, uint256 deadline) external onlySeraRole(EXECUTOR_ROLE_CACHED)
whenNotPaused {
    ...
    for (uint256 i = 0; i < matches.length;) {
        MatchData calldata m = matches[i];
        // Enforce: all taker orders belong to same user
        if (m.order0.user != takerUser) revert InvalidRoute();
        ...
    }
    ...
}
```

Recommendation

We recommend making slippage allocation explicitly role-aware at settlement time.

- Bind each calculated share component to the actual maker/taker recipient mapping (derived from route context or canonical role assignment), rather than to `order0/order1` payment channels.
- For standalone flows, enforce a canonical role ordering or reject ambiguous orientation to prevent order-position-dependent outcomes.

Alleviation

[Sera, 03/28/2026]:

The SOR logic is refactored to fix 11, 12, and 13.

[CertiK, 04/02/2026]:

Asymmetric slippage shares are applied in `_calculateSettlement` through fixed `order0` / `order1` payout channels (`executionValue0`, `executionValue1`). For direct `matchOrders` fills, both legs are validated as maker-style orders, but the entry point does not fix a single canonical maker-taker orientation before settlement. `EXECUTOR_ROLE` therefore chooses which signed order is supplied as `order0` versus `order1`.

When `makerShareBps` and `takerShareBps` differ and the match creates a positive spread, swapping that ordering changes which counterparty receives the more favorable user-facing portion of the spread, without changing the signed order payloads or limit prices. This is a value-allocation / fairness concern on the permissioned matching path. Routed legs use a stable convention (`order0` = taker, `order1` = maker); the issue is primarily about direct `matchOrders` where input ordering is not constrained.

Consider binding spread allocation to an unambiguous role or deterministic ordering or making the sharing rule symmetric so swapping the two valid orders cannot change final payouts unless that behavior is explicitly intended and documented.

[CertiK, 04/10/2026]:

In the current order matching model (where it is explicitly defined that `order0` is the taker and `order1` is the maker).

```
/// @notice Compute fees, spreads, update filled amounts, and return settlement results.
/// @dev Spread is divided into protocol, maker (order1), and taker (order0) shares.
///      In matchOrders, both sides are limit orders with no intrinsic role – the executor's
///      input ordering determines who receives which share. This is by design: the executor
///      is a trusted role. In settleRoutedLeg, order0 is always the SOR taker.
```

We analyzed the settlement flow and spread distribution logic for a single matched trade. From a conceptual standpoint, focusing on the `order0` side, the following relationship is expected to hold:

```
effectiveAmount0 = executionValue1 + protocolSpread0 + spreadToMaker0 + spreadToTaker0
```

Under an intuitive interpretation:

- `spreadToMaker0` is allocated to the maker
- `spreadToTaker0` is allocated to the taker

Then, on the `order0` (taker) side, the expected fund flow would be:

- **Taker actual payment:**

```
effectiveAmount0 - spreadToTaker0
```

- **Maker actual receipt:**

```
executionValue1 + spreadToMaker0 - protocolFee0
```

- **Treasury actual receipt:**

```
protocolSpread0 + protocolFee0
```

Under this model, the following holds:

```
taker deduction = maker increase + treasury increase
```

However, according to the current implementation (which distributes spread by manipulating payout values):

```
calc.executionValue1 = executionValue1 + spreadToTaker0;
calc.executionValue0 = executionValue0 + spreadToMaker1;
```

The actual fund flow becomes:

Taker → Maker:

```
executionValue1 + spreadToTaker0 - protocolFee0
```

• **Taker → Treasury:**

```
protocolFee0 + protocolSpread0
```

Therefore, the actual taker deduction is:

```
executionValue1 + spreadToTaker0 + protocolSpread0 = effectiveAmount0 -
spreadToMaker0
```

Although the current implementation still preserves total value conservation (i.e., taker deduction equals maker + treasury increase), there is an inconsistency between the spread attribution and its intuitive interpretation.

We would like to further confirm with the project team the exact semantic definition of `spreadToMaker` and `spreadToTaker`. If the design goal is to align spread attribution with its naming (i.e., `spreadToTaker` effectively benefits the taker, and `spreadToMaker` effectively benefits the maker), then the current implementation deviates from this intuitive interpretation. In that case, the settlement logic may need to be adjusted to avoid potential misinterpretation or incorrect spread distribution.

[Sera, 04/17/2026]:

The current implementation is semantically correct. `spreadToTaker0` benefits the taker by reducing what they pay (via `executionValue1 +=`), and `spreadToMaker1` benefits the maker by increasing what they receive (via `executionValue0 +=`).

The suggested swap (`spreadToMaker0` to `executionValue1`, `spreadToTaker1` to `executionValue0`) would actually invert the intended allocation, causing maker-designated shares to benefit the taker and vice versa.

The real issue is the asymmetric share configuration. We will align `makerShareBps == takerShareBps` to ensure order-position-independent outcomes in `matchOrders()`.

[CertiK, 04/17/2026]:

We have a different interpretation of the following point:

```
spreadToTaker0 benefits the taker by reducing what they pay (via executionValue1 +=), and spreadToMaker1 benefits the maker by increasing what they receive (via executionValue0 +=).
```

Under the case where `order0 = taker` and `order1 = maker`, looking at the token0 side:

- `executionValue1` is the main payment flow from order0 to order1 (debited from order0, credited to order1).
- The code applies `executionValue1 += spreadToTaker0`, so this portion is explicitly received by the maker.
- Meanwhile, `totalSpread0 = protocolSpread0 + spreadToMaker0 + spreadToTaker0`, and the taker's total cost is: `executionValue1 + spreadToTaker0 + protocolSpread0 = effectiveAmount0 - spreadToMaker0`, meaning the taker effectively benefits from `spreadToMaker0` via paying less (an implicit rebate).

Summary:

- The taker benefits from `spreadToMaker0` (implicit rebate)
- The maker benefits from `spreadToTaker0` (explicit payout increase)

Additionally, we agree that setting `makerShareBps == takerShareBps` can help mitigate the role ambiguity issue. However, in the current implementation, `makerShareBps` is set to 0.

```
slippageShares = SlippageShare({
  makerShareBps: 0,
  takerShareBps: 5000,
  protocolShareBps: 5000, // Default to 50% protocol capture natively
  totalBps: 10000 // 100% denominator
});
```

[CertiK, 04/17/2026]:

The team updated the default configuration to `makerShareBps = takerShareBps = 2500`, which mitigates order-position sensitivity in practice. However, if maker and taker shares diverge again, the same role/ordering sensitivity issue will reappear.

Changes have been reflected in the commit hash: [9700d330aa69ad695cfd42441a1393e887441d78](#).

[Sera, 04/20/2026]:

We've swapped the maker/taker role in the latest commit. Changes have been reflected in the commit hash: [c56510f76c5935f5f912058ab910e015e3af0f40](#).

[CertiK, 04/23/2026]:

The team has set the initial `slippageShares` to `maker=0, taker=0, protocol=10000`. Under this default configuration, all positive slippage (spread) is fully allocated to the protocol. The admin can still adjust the distribution ratios later via `setSlippageShares()`.

Changes have been reflected in the commit hash: [dd974081b50c4cb378ca5312611ef5574c56b4d9](#).

SFO-13 | Unbound Partial Execution Can Sweep Taker Funds To Treasury

Category	Severity	Location	Status
Logical Issue	Medium	src/SeraSOR.sol (03/23-738f0a8): 52	Resolved

Description

In `SeraSOR` contract, the `executeRoute(...)` function handles multi-leg routed trades and tracks temporary balances in `transientTokens` and `transientAmounts`. The issue arises because the signed route does not bind executor-provided `matchAmount0` and `matchAmount1`. Any leftover transient balance at the end of a route is automatically swept to the protocol treasury via `sweepTransientToProtocol()`.

This can result in genuine taker-owned funds remaining in Sera after partial or truncated route execution and being transferred to `treasury` instead of being refunded or reverted. It affects both unused `matches[0].order0.initialDepositAmount` and intermediate assets held when `recipient == address(sera)`.

The problem stems from two key factors:

1. Route authorization is incomplete: `_computeRouteHash()` only commits `order0` fields (user, expiration, feeBps, recipient, fromToken, toToken, fromAmount, toAmount, initialDepositAmount, bytes32(0), uuid) and does not include `matchAmount0` or `matchAmount1`. `_validateRouteSignature()` therefore accepts partially executed routes.
2. Per-leg validation is local and does not enforce full route-wide consumption. `_validateOrderCommon()` only checks `matchAmount` against order limits, not that the entire route or initial deposit is used.

As a result, partial fills leave residual taker-owned tokens in `transientAmounts`, which are later treated as protocol-owned and swept to `treasury`. This can happen in both intermediate outputs and wallet-prefunded routes where only part of the pre-funded amount is actually used. Users can lose funds repeatedly if the same partially fillable signed route is reused.

Proof of Concept

This PoC simulates a route where the taker signs and authorizes a wallet pull of the full `initialDepositAmount`, but the executor only submits a partial `matchAmount` on-chain (here 50%). In the end, the unused prefunded balance remains tracked as transient inventory and is swept to the protocol treasury at route completion, so the user neither receives it back nor has it applied to the executed trade.


```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.24;

import "forge-std/Test.sol";
import "../src/Sera.sol";
import "../src/SeraSOR.sol";
import "../src/mock/MockStableCoin.sol";
import "../TestHelper.sol";

contract PoCInitialDepositSweep is TestHelper {
    Sera public sera;
    SeraSOR public sor;
    MockStableCoin public usdc;
    MockStableCoin public eth;

    address public owner;
    address public executor;
    address public taker;
    uint256 public takerPK;
    address public maker;
    uint256 public makerPK;

    function setUp() public {
        owner = makeAddr("owner");
        executor = makeAddr("executor");
        (taker, takerPK) = makeAddrAndKey("taker");
        (maker, makerPK) = makeAddrAndKey("maker");

        usdc = new MockStableCoin("USDC");
        eth = new MockStableCoin("ETH");

        sera = _deploySera(owner);
        sor = new SeraSOR(address(sera));

        vm.startPrank(owner);
        _whitelistToken(sera, address(usdc), true, 1);
        _whitelistToken(sera, address(eth), true, 1);
        sera.grantRole(sera.EXECUTOR_ROLE(), executor);
        sera.grantRole(sera.EXECUTOR_ROLE(), address(sor));
        sera.grantRole(sera.EXECUTOR_ROLE_CACHED(), address(sor));
        sera.setTrustedRouter(address(sor));
        vm.stopPrank();
    }

    function _finalizeRouteBindings(MatchData[] memory matches) internal pure
    returns (bytes32) {
        bytes memory packed = new bytes(matches.length * 32);
        for (uint256 i = 0; i < matches.length; i++) {
            Order memory p = matches[i].order0;
```

```
p.routeHash = bytes32(0);
bytes32 structHash = keccak256(
    abi.encode(
        ORDER_TYPEHASH,
        p.user,
        p.expiration,
        p.feeBps,
        p.recipient,
        p.fromToken,
        p.toToken,
        p.fromAmount,
        p.toAmount,
        p.initialDepositAmount,
        p.routeHash,
        p.uuid
    )
);
assembly {
    let offset := add(add(packed, 32), mul(i, 32))
    mstore(offset, structHash)
}
}
// Route hash = keccak(order0 struct hashes only). matchAmount0/1 are NOT
committed.
bytes32 routeHash = keccak256(packed);
for (uint256 i = 0; i < matches.length; i++) {
    matches[i].order0.routeHash = routeHash;
}
return routeHash;
}

function test_PoC_initialDeposit_partialMatch_unusedHalfSweptToTreasury() public
{
    // Taker will authorize pulling this much from wallet at route start (see
    executeRoute).
    uint256 prefund = 1000 ether;
    usdc.mint(taker, prefund);
    vm.prank(taker);
    usdc.approve(address(sor), prefund);

    // Maker liquidity for a full 1000 USDC ↔ 10 ETH leg (we will only use half
    on-chain).
    _mintAndDeposit(maker, address(eth), 10 ether, sera);

    Order memory takerOrder = Order({
        user: taker,
        fromToken: address(usdc),
        toToken: address(eth),
        fromAmount: 1000 ether,
        toAmount: 10 ether,
```

```
// Signed into route hash: SOR always pulls this full amount from wallet
before legs.
    initialDepositAmount: prefund,
    feeBps: 0,
    recipient: taker,
    expiration: uint48(block.timestamp + 1 days),
    uuid: 4242,
    routeHash: bytes32(0)
});

Order memory makerOrder = Order({
    user: maker,
    fromToken: address(eth),
    toToken: address(usdc),
    fromAmount: 10 ether,
    toAmount: 1000 ether,
    initialDepositAmount: 0,
    feeBps: 0,
    recipient: maker,
    expiration: uint48(block.timestamp + 1 days),
    uuid: 4343,
    routeHash: bytes32(0)
});

MatchData[] memory matches = new MatchData[](1);
matches[0] = MatchData({
    order0: takerOrder,
    signature0: bytes(""),
    // 50% on-chain fill – matchAmounts are executor-supplied and not in
routeHash.
    matchAmount0: 500 ether,
    order1: makerOrder,
    signature1: _signOrder(makerPK, makerOrder, sera),
    matchAmount1: 5 ether
});

bytes32 routeHash = _finalizeRouteBindings(matches);
bytes memory routeSig = _signRoute(takerPK, routeHash, sera);

address trsy = sera.treasury();
uint256 treasuryBefore = sera.vault().balanceOf(address(usdc), trsy);

// executeRoute: pull full prefund → settle leg → sweep nonzero transient to
treasury .
vm.prank(executor);
sor.executeRoute(matches, routeSig, type(uint256).max);

// Taker wallet empty: full prefund was moved; trade output is only half the
signed leg.
```

```
    assertEq(usdc.balanceOf(taker), 0, "wallet: full initialDepositAmount was
pulled");
    assertEq(eth.balanceOf(taker), 5 ether, "taker: only 50% leg output");
    assertEq(usdc.balanceOf(maker), 500 ether, "maker: only 50% input
received");

    // Other half of prefund: not returned to taker – vault credits treasury via
sweepTransientToProtocol.
    assertEq(
        sera.vault().balanceOf(address(usdc), trsy),
        treasuryBefore + 500 ether,
        "unused prefund swept to treasury"
    );
    assertEq(usdc.balanceOf(address(sera)), 0, "no USDC dust left on Sera");
}
}
```

Recommendation

Consider not treating leftover transient balances as protocol-owned by default. Ensure unused funds are either refunded to the taker or cause a reversal. Enforce full route-level accounting, consume prefunded amounts only as authorized, and clearly distinguish protocol fees from user funds before sweeping to treasury. For partial fills, define explicit route-level semantics, including leg execution limits, intermediate output bounds, and unused fund handling.

Alleviation

[CertiK, 04/02/2026]:

The team refactored the SOR logic, and the issue does not exist in commit [573a50dced379594ab5dbca5cc10a86802b716da](#).

SFO-05 Same-Token Matching Weakens Withdrawal Control Boundaries

Category	Severity	Location	Status
Logical Issue	Minor	src/Sera.sol (03/23-738f0a8): 225	Resolved

Description

The `matchOrders()` function only enforces cross-order token symmetry and does not reject orders where `fromToken == toToken`. This allows reciprocal same-token orders to settle through the trading path and send funds to external `recipient` addresses, functionally overlapping with withdrawal behavior instead of strictly trade-only behavior. Although this scenario depends on privileged executor participation and valid signed orders, it is still recommended to add completeness checks to fully eliminate this class of edge-case execution.

```
function matchOrders(MatchData calldata _match, uint256 deadline) external
onlyRole(EXECUTOR_ROLE) whenNotPaused nonReentrant {
    ...
    // Token symmetry
    @> if (_match.order0.fromToken != _match.order1.toToken ||
_match.order1.fromToken != _match.order0.toToken) revert TokenMismatch();
    _executeMatch(_match, orderHash0, orderHash1);
}
```

Recommendation

It is recommended to add explicit validation to disallow same-token orders in matching paths (e.g., require `fromToken != toToken`), and keep withdrawal-like fund exits strictly within dedicated withdrawal flows.

Alleviation

[Sera, 03/28/2026]:

Issue acknowledged. Changes have been reflected in the commit hash: [a52110d54faeab85615475f79bf587fbff2a11d4](#).

[CertiK, 04/03/2026]:

The `settleRoutedLeg()` function does not apply the same trade-shape guards as `matchOrders()`. In `matchOrders()`, the protocol rejects same-token legs via `SameTokenMatch` (`order0.fromToken == order0.toToken`) and rejects self-matching via `SelfMatch` when the two order hashes are identical, explicitly to prevent `filledAmount` from being updated twice for the same order hash and to block withdrawal-like same-asset flows.

The routed path (`settleRoutedLeg`) only enforces cross-order token symmetry (`TokenMismatch`) and omits both `SameTokenMatch` and `SelfMatch`, so behavior is inconsistent with direct matching and can admit routed legs that

`matchOrders()` would revert.

Consider adding the same requirement in the `settleRoutedLeg()` function.

[Sera, 04/09/2026]:

Issue acknowledged. Changes have been reflected in the commit hash: [905e7f7ba3a669fa0b2e393afeb289e5189c04f6](#).

SFO-06 | Identical Order Self-Match Corrupts Fill Accounting

Category	Severity	Location	Status
Logical Issue	● Minor	src/Sera.sol (03/23-738f0a8): 218~219	● Resolved

Description

The `matchOrders()` function validates `order0` and `order1` independently but does not enforce `orderHash0 != orderHash1`. If the same signed order is used on both sides (in a same-token-compatible setup), `filledAmount` can be incremented twice in one call, causing premature exhaustion or over-advancement of a single order's fill state. Although this scenario depends on privileged executor participation, adding complete anti-self-match validation is recommended to fully prevent this accounting inconsistency.

```
function matchOrders(MatchData calldata _match, uint256 deadline) external
onlyRole(EXECUTOR_ROLE) whenNotPaused nonReentrant {
    if (block.timestamp > deadline) revert MatchExpired();
    bytes32 orderHash0 = SeraLib.getOrderHashCalldata(_match.order0);
    bytes32 orderHash1 = SeraLib.getOrderHashCalldata(_match.order1);
    ...
}
```

Recommendation

It is recommended to require `orderHash0 != orderHash1` in matching and optionally add a defensive invariant check to prevent any single order hash from being updated twice within one settlement transaction.

Alleviation

[Sera, 03/28/2026]:

Issue acknowledged. Changes have been reflected in the commit hash: [b96c0493211b1bf6e7e00cff75ce20f1ae52c05e](#).

[CertiK, 04/03/2026]:

The `settleRoutedLeg()` function does not apply the same trade-shape guards as `matchOrders()`. In `matchOrders()`, the protocol rejects same-token legs via `SameTokenMatch` (`order0.fromToken == order0.toToken`) and rejects self-matching via `SelfMatch` when the two order hashes are identical, explicitly to prevent `filledAmount` from being updated twice for the same order hash and to block withdrawal-like same-asset flows.

The routed path (`settleRoutedLeg`) only enforces cross-order token symmetry (`TokenMismatch`) and omits both `SameTokenMatch` and `SelfMatch`, so behavior is inconsistent with direct matching and can admit routed legs that `matchOrders()` would revert.

Consider adding the same requirement in the `settleRoutedLeg()` function.

[Sera, 04/09/2026]:

Issue acknowledged. Changes have been reflected in the commit hash: [905e7f7ba3a669fa0b2e393afeb289e5189c04f6](#).

[CertiK, 04/18/2026]:

Currently, matching includes validation of the same order hash and same token to prevent matching the same order.

However, is it still possible for two different orders from the same user to be matched, i.e., `order0.user == order1.user` ? If this behavior is not intended, it would be better to explicitly prohibit it in the validation logic.

[Sera, 04/26/2026]:

We considered explicit on-chain enforcement and concluded it is not warranted. Self-matching with a single wallet has no economic upside: the user simultaneously debits and credits the same balance while paying protocol fees and spread share on both sides, producing a strictly negative outcome. Even under a compromised Web2 layer, an attacker cannot extract value through self-matching, since no net token movement occurs in their favor.

As a defense-in-depth measure, our Web2 order intake layer rejects submissions where `order0.user == order1.user`, ensuring such configurations do not reach the executor under normal operation. We have therefore omitted the on-chain check to avoid the additional gas overhead on every match.

SFO-08 | Missing Zero Address Validation

Category	Severity	Location	Status
Volatile Code	Minor	src/Vault.sol (03/23-738f0a8): 25	Resolved

Description

The `Vault` contract assigns `DEFAULT_ADMIN_ROLE` to `initialAdmin` but does not validate against `address(0)` when deploying. If the contract is deployed with the zero address, the admin role is effectively assigned to an unusable account.

```
constructor(address initialAdmin) {  
    _grantRole(DEFAULT_ADMIN_ROLE, initialAdmin);  
}
```

As a result, administrative control becomes permanently inaccessible—no one can grant `TRADER_ROLE` or recover admin privileges, which will cause the `Vault` contract to become permanently non-functional instead of failing fast at deployment due to invalid configuration.

Recommendation

It is recommended to add a zero address check to avoid this scenario.

Alleviation

[Sera, 03/28/2026]:

Issue acknowledged. Changes have been reflected in the commit hash: [64c1bd5738589643c101bf7a8a1ba6e20007544c](#).

SFO-11 Non-Final Routed Legs Do Not Enforce `order0.recipient == address(sera)`

Category	Severity	Location	Status
Design Issue	Minor	src/Sera.sol (03/23-738f0a8): 402~403; src/SeraSOR.sol (03/23-738f0a8): 90	Resolved

Description

In `SeraSOR.executeRoute`, an intermediate leg only tops up the router's in-memory transient balance when `holdTakerOutput` is true, i.e., when it is not the last leg and `m.order0.recipient == address(sera)`. In that case the leg's output stays (or is accounted for) inside the route execution path so `_consumeTransientBalance` can use it.

```
bool holdTakerOutput = !isLastLeg && (m.order0.recipient == address(sera));
```

If an intermediate leg instead sets `m.order0.recipient` to any address other than `address(sera)` (for example the taker's EOA), settlement delivers that leg's output to that recipient via the normal vault payout path. That amount is not added to the transient table for the following leg.

```
function _settleRoutedLegInternal(MatchData calldata _match, bytes32 takerHash,
bytes32 makerHash, uint256 takerVaultPull, bool holdTakerOutput) internal returns
(uint256 takerReceives) {
    ...
    address takerRecipient = holdTakerOutput ? address(this) :
_match.order0.recipient;
    _executeVaultSettlement(_match.order1.user, _match.order0.user,
takerRecipient, makerFromToken, takerReceives, calc.protocolTake1);
    ...
}
```

The next leg attempts to source the taker's input by first consuming transient balances, then pulling the remainder from the taker's vault. However, tokens received in the taker's wallet from the previous leg are not automatically reused for subsequent legs. While the vault can cover any shortfall, enforcing intermediate leg receipts to be routed to Sera (i.e., `recipient = sera`) provides stronger guarantees of execution success by ensuring funds remain in the expected flow.

Recommendation

We would like to confirm with the team whether this approach aligns with the intended design.

Alleviation

[Certik, 04/02/2026]:

The team refactored the SOR logic, and the issue does not exist in commit [573a50dced379594ab5dbca5cc10a86802b716da](#).

SFO-15 | Unnormalized `v` In 65-Byte Permit Signatures May Break EIP-2612 Integration

Category	Severity	Location	Status
Logical Issue	Minor	src/SeraSOR.sol (04/02-573a50d): 232~254	Resolved

Description

In `SeraSOR._executePermit`, the 65-byte signature path decodes `r`, `s`, and `v` from calldata and passes `v` directly into `IERC20Permit.permit` without normalizing `0/1` to `27/28`. In contrast, the 64-byte (EIP-2098) path reconstructs a canonical `v`.

```
function _executePermit(address token, address owner, address spender, uint256
amount, uint256 deadline, bytes calldata signature) internal {
    uint8 v;
    bytes32 r;
    bytes32 s;
    if (signature.length == 65) {
        assembly {
            r := calldataload(signature.offset)
            s := calldataload(add(signature.offset, 0x20))
            v := byte(0, calldataload(add(signature.offset, 0x40)))
        }
    } else if (signature.length == 64) {
        assembly {
            let vs := calldataload(add(signature.offset, 0x20))
            r := calldataload(signature.offset)
            s := and(vs,
0x7fffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff)
            v := add(shr(255, vs), 27)
        }
    } else {
        revert Sera.InvalidSignatureLength();
    }

    try IERC20Permit(token).permit(owner, spender, amount, deadline, v, r, s) {}
    catch {}
}
```

Some EIP-2612 token implementations only accept canonical `v` values. As a result, signatures using `v = 0/1` may cause `permit` to revert. This revert is silently swallowed via `try/catch`, potentially leaving the allowance unset and causing subsequent `safeTransferFrom` calls to fail when no prior allowance exists.

The impact depends on the signature encoding, the token's `permit` implementation, and whether sufficient allowance already exists. However, the inconsistent handling between signature formats introduces implicit failure modes and integration risk.

Recommendation

Normalize `v` in the 65-byte signature path (map to `27/28`) to ensure consistency with the 64-byte path. If execution relies on `permit` to establish allowance, avoid silently ignoring failures—either revert explicitly or verify that sufficient allowance exists after a failed permit attempt.

Alleviation

[Sera, 04/09/2026]:

Issue acknowledged. Changes have been reflected in the commit hash: [905e7f7ba3a669fa0b2e393afeb289e5189c04f6](#).

SFO-16 `maxInputAmount` Is Enforced On Gross Staged Input Rather Than Net Taker Spend

Category	Severity	Location	Status
Logical Issue	● Minor		● Acknowledged

Description

In `SeraSOR.executeIntent()`, `takerInputCost` accumulates the initial deposit (`initialDepositAmount`) together with each leg's `takerVaultPull` for the input token. The final check enforces `takerInputCost <= intent.maxInputAmount`.

However, during routed settlement, `Sera._settleRoutedLegInternal` may withdraw less than the staged `takerVaultPull` and may also credit token-side surplus (e.g., `physicalSurplus`) back to the taker's vault. As a result, `takerInputCost` reflects gross staged input, while the taker's actual net consumption of the input token can be lower.

This mismatch means that routes that are economically within the user's intended spend can still revert due to exceeding `maxInputAmount` at the gross level. The issue is more pronounced when surplus-return paths apply (e.g., non-zero `makerShareBps`). This is primarily a guard semantics / UX inconsistency, where users may be forced to authorize a higher limit than what is ultimately consumed.

Recommendation

Enforce `maxInputAmount` based on the taker's realized net input-token consumption after all settlement steps (including partial withdrawals and surplus credits), or maintain gross staging for internal accounting while introducing a separate variable for the user-facing limit check.

Alleviation

[CertiK, 04/10/2026]:

In simple terms, within the `executeIntent()` function, the initial transferred amount and the per-leg planned vault pulls (`takerVaultPull`) are aggregated into `takerInputCost`, which is then compared against `intent.maxInputAmount`.

However, during the actual settlement of each leg (`sera.settleRoutedLeg`), `Sera` may pull less from the vault or refund excess input tokens back to the user's vault ledger, i.e., `actualPull < takerVaultPull`, and additional refunds may also occur via `vault.creditLedger()`.

```
function _settleRoutedLegInternal(MatchData calldata _match, bytes32 takerHash,
bytes32 makerHash, uint256 takerVaultPull, bool holdTakerOutput, uint256
effectiveMatchAmount0) internal returns (uint256 takerReceives) {
    ...
    if (neededFromTaker > transientPhysical) {
        ...
        if (actualPull > 0)
        @> vault.withdraw(_match.order0.user, takerFromToken, actualPull, address(this));
    } else if (transientPhysical > neededFromTaker) {
        // Surplus: The transient tokens alone exceed the cost.
        // No vault pull is needed, and we return the surplus transient physical
        tokens securely to their vault ledger
        uint256 physicalSurplus = transientPhysical - neededFromTaker;
        IERC20(takerFromToken).safeTransfer(address(vault), physicalSurplus);
        vault.creditLedger(_match.order0.user, takerFromToken, physicalSurplus);
    }
}
```

As a result, when the final check is performed against `takerInputCost`, the transaction may still be considered over the limit even if the net consumption does not exceed the user's intended cap.

This issue primarily affects the consistency of limit semantics and user experience. It is not a typical overcharging issue, but rather a mismatch between the meaning of the signed limit and the on-chain validation logic. If the team considers this behavior to be an acceptable design choice, the issue can be disregarded.

[Sera, 04/20/2026]:

Vault pull is not supported in our current front end. Even if our Web2 were compromised, no actual loss of funds or value would happen, as both vault and wallet tokens are owned by the user.

SFO-17 Intent Replay Semantics Depend On Single-Versus-Multi Router Deployments

Category	Severity	Location	Status
Design Issue	● Minor	src/SeraSOR.sol (04/02-573a50d): 171~180	● Resolved

Description

Intent signatures are validated using a digest produced via `Sera.getIntentDigest(...)`, which binds the payload to the `Sera` contract's EIP-712 domain. Consumption of a given `(user, uuid)` is currently tracked in `SeraSOR` local storage (`isIntentUuidUsed`). If the protocol only ever uses one router instance per `Sera`, that design is internally consistent: an intent cannot be executed twice on that router after `uuid` is marked used.

```
function _validateAndConsumeIntent(bytes calldata signature, IntentParams
calldata p) internal returns (address takerUser) {
    // Validate EIP-712 signature
    bytes32 digest = sera.getIntentDigest(p.inputToken, p.outputToken,
p.maxInputAmount, p.minOutputAmount, p.recipient, p.initialDepositAmount, p.uuid,
p.deadline);
    takerUser = SoladyECDSA.recover(digest, signature);
    if (takerUser == address(0)) revert Sera.InvalidSignature();

    // Per-user replay protection: each uuid can only execute once per user
    if (isIntentUuidUsed[takerUser][p.uuid]) revert IntentAlreadyUsed();
    isIntentUuidUsed[takerUser][p.uuid] = true;
}
```

If the product roadmap includes additional `SeraSOR` deployments that reuse the same `Sera` and the same digest function, the same signed intent can remain cryptographically valid on a new router whose `isIntentUuidUsed` mapping is fresh, which may allow a second execution of the same signed parameters (subject to `deadline`, balances, approvals, and executor constraints). Whether that is acceptable is a deployment and security-model choice, not an automatic user bug.

Recommendation

We would like to confirm the intended operational model. The design assumes either a single active `SeraSOR` (or other router) instance per `Sera`, or multiple router instances may be deployed over time (e.g., upgrades, different chains, or regional deployments) while sharing the same `Sera` and digest.

Alleviation

[Sera, 04/09/2026]:

Issue acknowledged. Changes have been reflected in the commit hash: [905e7f7ba3a669fa0b2e393afeb289e5189c04f6](#).

SFO-18 | Delayed Withdrawal Not Bound To Balance Weakens Cooldown Mechanism

Category	Severity	Location	Status
Logical Issue	● Minor	src/Sera.sol (04/09-905e7f7): 195	● Resolved

Description

The `emergencyWithdraw()` function only records `requestBlock` and `amount`, without binding the withdrawal request to the user's actual vault balance at the time of request creation. Once matured, the request is treated as a reusable maximum withdrawal allowance rather than a delayed release of specific funds. As a result, a user can wait through a single delay period and then immediately withdraw subsequently credited balances of the same token (e.g., from new deposits or settlement proceeds). This design bypasses the intended cooldown window, allowing newly added funds to be withdrawn without delay, thereby weakening the synchronization mechanism and potentially reintroducing ghost liquidity and settlement grieving risks.

```
function emergencyWithdraw(address token, uint256 amount) external nonReentrant
{
    if (token == address(0)) revert InvalidToken(token);
    if (amount == 0) revert InvalidAmount();

    WithdrawRequest storage request = withdrawRequests[msg.sender][token];

    // If no request exists OR the request has expired, treat it as a new
    request
    if (request.requestBlock == 0 || block.number > request.requestBlock +
    WITHDRAW_EXPIRATION_BLOCKS) {
        request.requestBlock = block.number;
        request.amount = amount;
        emit WithdrawRequested(msg.sender, token, amount, block.number);
        return;
    }

    uint256 blocksPassed = block.number - request.requestBlock;
    if (blocksPassed < WITHDRAW_DELAY_BLOCKS) revert WithdrawNotReady();
    if (amount > request.amount) revert AmountMismatch();

    delete withdrawRequests[msg.sender][token];

    vault.withdraw(msg.sender, token, amount, msg.sender);
    emit Withdraw(token, msg.sender, amount);
}
```

I Recommendation

We would like to confirm whether this behavior is aligned with the intended design. If not, consider binding the withdrawal request to the user's balance at the time of request creation to ensure newly credited funds are still subject to the delay period.

I Alleviation

[Sera, 04/17/2026]:

The team heeded the advice and resolved the issue in commit [9700d330aa69ad695cfd42441a1393e887441d78](#).

SFO-01 | Permit Function Doesn't Comply With DAI Token

Category	Severity	Location	Status
Design Issue	● Informational	src/Sera.sol (03/23-738f0a8): 143	● Acknowledged

Description

Several contracts in this project support allowance grants via permit signatures. However, there is a compatibility issue with DAI's non-EIP-2612 permit interface.

DAI at `0x6B175474E89094C44Da98b954EedeAC495271d0F` is a standard ERC-20 token, but its permit function uses Maker's legacy signature format:

```
permit(address holder, address spender, uint256 nonce, uint256 expiry, bool allowed, uint8 v, bytes32 r, bytes32 s)
```

This differs from the EIP-2612-style interface expected by the current implementation:

```
permit(address owner, address spender, uint256 value, uint256 deadline, uint8 v, bytes32 r, bytes32 s)
```

Because the parameter schema and signed payload semantics are different, a DAI permit cannot be consumed through the EIP-2612 call path. As a result, permit-based authorization for DAI will fail. In this codebase, that failure is swallowed by `try/catch`, so the transaction only succeeds if allowance is already sufficient; otherwise, the subsequent deposit step fails due to insufficient allowance.

Recommendation

To address the compatibility issue with the DAI token's unique permit function, a targeted solution should be implemented if the `DAI` token is within the scope of usage. Specifically, when interacting with the `DAI` token, the contract should utilize a specialized version of the permit function that accommodates the DAI-specific nonce parameter.

Alleviation

[Sera, 03/28/2026]:

We will not support DAI in the short term for gasless deposits, and as such, it is a non-issue.

SFO-07 | Partially Filled Orders Ignore Later Whitelist/Min-Amount Policy Updates

Category	Severity	Location	Status
Logical Issue	● Informational	src/Sera.sol (03/23-738f0a8): 324~325	● Acknowledged

Description

The `_validateOrderCommon()` function enforces `tokenConfigs[order.fromToken]` checks (`isWhitelisted` and `minAmount`) only when `filledAmount[orderHash] == 0`.

```
function _validateOrderCommon(Order calldata order, bytes32 orderHash, uint256
matchAmount) internal view returns (uint256 filled) {
    ...
    // First time validations, skipped for subsequent fills to save gas
    if (filled == 0) {
        if (order.fromAmount == 0 || order.toAmount == 0) revert
InvalidAmount();
        if (order.expiration > block.timestamp + MAX_EXPIRATION) revert
OrderExpirationTooLong();
        if (order.feeBps > BPS_DENOMINATOR) revert InvalidFee(); // Not more
than 100% fee
        TokenConfig memory config = tokenConfigs[order.fromToken];
        @> if (!config.isWhitelisted) revert TokenNotWhitelisted(order.fromToken);
        @> if (order.fromAmount < config.minAmount) revert
AmountBelowMinimum(order.fromAmount, config.minAmount);
    }
}
```

As a result, after the first successful fill of an order, subsequent fills for the same `orderHash` no longer re-evaluate later token-policy changes (such as de-whitelisting or raising minimum amount). This behavior applies to both `matchOrders()` and `settleRoutedLeg()` because both execution paths reuse `_validateOrderCommon()`.

Recommendation

We noticed that this issue is mentioned in the FAQ document, but we would like to reconfirm it with the team. It is recommended to move whitelist/min checks outside the `filled == 0` branch so they run on every fill.

Alleviation

[Sera, 03/25/2026]:

This isn't an issue. If a whitelist check is added to every fill, then gas is more expensive per fill.

Furthermore, our frontend will limit this. If our executor is compromised, trading delisted tokens is not the right course of action. The right course of action is to pause Sera since the executor key is compromised.

SFO-09 | Inconsistent Blacklist Enforcement

Category	Severity	Location	Status
Inconsistency, Logical Issue	● Informational	src/Vault.sol (03/23-738f0a8): 72	● Acknowledged

Description

In the `Vault` contract, it applies blacklist checks inconsistently across ledger-credit paths. The `creditLedger(...)` function rejects blacklisted recipients, while the `transferLedger(...)` function does not enforce blacklist checks on `toUser`.

```
function creditLedger(address user, address token, uint256 expectedAmount)
external override nonReentrant onlyRole(TRADER_ROLE) {
    if (user == address(0)) revert ZeroAddress();
    if (blacklisted[user]) revert BlacklistedUser(user);
    if (expectedAmount == 0) revert ZeroAmount();
    balances[token][user] += expectedAmount;
    trackedBalance[token] += expectedAmount;
    emit Deposited(token, user, expectedAmount);
}
// Trader-only withdraw to arbitrary recipient. user == to is self-withdraw.

function transferLedger(address fromUser, address toUser, address token, uint256
amount) external override nonReentrant onlyRole(TRADER_ROLE) {
    if (toUser == address(0)) revert ZeroAddress();
    if (amount == 0) revert ZeroAmount();
    uint256 fromBal = balances[token][fromUser];
    if (fromBal < amount) revert InsufficientBalance();
    balances[token][fromUser] = fromBal - amount;
    balances[token][toUser] += amount;
    // trackedBalance[token] does not change because total vault TVL remains the
same
    // Emit withdrawal and deposit events to keep standard subgraph indexers
working
    emit Withdrawn(token, fromUser, amount);
    emit Deposited(token, toUser, amount);
}
```

As a result, protocol fee accounting can behave differently depending on the settlement route:

- Direct match fee flow uses `transferLedger(..., treasury, ...)` and can succeed without blacklist gating on recipient.
- Routed-leg and transient-sweep fee flow uses `creditLedger(treasury, ...)` and reverts if `treasury` is blacklisted.

This creates path-dependent behavior for the same protocol fee destination and weakens policy consistency guarantees.

While blacklisting `treasury` is an uncommon operation, the core issue is incomplete validation in `transferLedger()`, not the rarity of that configuration.

Recommendation

Consider a unified blacklist policy across ledger mutation functions. At minimum, add recipient blacklist validation to `transferLedger` (e.g., reject blacklisted `toUser`) so behavior matches `creditLedger`. Optionally define and document whether sender-side blacklist checks should also be enforced for internal transfers.

Alleviation

[Sera, 03/25/2026]:

As credit and transfer ledgers are both only called by `Sera.sol`, which already checks for blacklisting, there's no reason to do a double check, which incurs more gas.

[CertiK, 04/02/2026]:

We consider this a reasonable and acceptable design choice, provided that `TRADER_ROLE` is strictly confined to `Sera` (or an equivalently trusted module), and that blacklist policies are consistently enforced across all code paths prior to any `creditLedger` / `transferLedger` calls. Under this model, using `Sera` as the single validation layer helps avoid redundant checks and reduces gas costs.

At the same time, adding corresponding blacklist checks on critical parameters within `Vault` (e.g., validating `toUser` in `transferLedger`, consistent with existing checks in `creditLedger`) would introduce only a small and predictable overhead while further strengthening defense in depth in case upstream logic or the permission model evolves.

SFO-14 | Order Lifetime Tied To Execution Time Instead Of Signing Time

Category	Severity	Location	Status
Design Issue	● Informational	src/Sera.sol (04/02-573a50d): 395	● Acknowledged

Description

`OrderExpirationTooLong` uses `order.expiration > block.timestamp + MAX_EXPIRATION`, so it only caps remaining lifetime at the moment of execution. It does not enforce “at most `MAX_EXPIRATION` after the order was signed.” A user can therefore sign an order expiring far in the future; it may fail early, but once the remaining window falls within one year of `block.timestamp` the same signature becomes valid again. That allows long-lived signatures and can diverge from comments that describe a one-year maximum order lifetime from signing.

```
function _validateOrderCommon(Order calldata order, bytes32 orderHash, uint256
matchAmount) internal view returns (uint256 filled) {
    if (matchAmount == 0) revert InvalidAmount();

    if (order.expiration <= block.timestamp) revert OrderExpired();
    if (vault.isBlacklisted(order.user)) revert
IVault.BlacklistedUser(order.user);

    filled = filledAmount[orderHash];

    if (filled + matchAmount > order.fromAmount) revert
OrderFilledAmountExceeded();

    // First time validations, skipped for subsequent fills to save gas
    if (filled == 0) {
        if (order.fromAmount == 0 || order.toAmount == 0) revert
InvalidAmount();
        @> if (order.expiration > block.timestamp + MAX_EXPIRATION) revert
OrderExpirationTooLong();
        if (order.feeBps > BPS_DENOMINATOR) revert InvalidFee(); // Not more
than 100% fee

        TokenConfig memory config = tokenConfigs[order.fromToken];
        if (!config.isWhitelisted) revert TokenNotWhitelisted(order.fromToken);
        if (order.fromAmount < config.minAmount) revert
AmountBelowMinimum(order.fromAmount, config.minAmount);
    }
}
```

Recommendation

Consider binding `MAX_EXPIRATION` to a signing timestamp in the signed payload (e.g. require `expiration - signedAt <= MAX_EXPIRATION`), or update comments and docs to state clearly that the limit applies to remaining lifetime at execution, not total lifetime since signing.

■ Alleviation

[Sera, 04/09/2026]:

Caps remaining lifetime at execution, not total lifetime since signing. A signature with a distant expiration may initially fail this check but become acceptable once `block.timestamp` advances close enough — this is by design.

SFO-20 | Partial-Fill Rounding Behavior Under Fragmented Execution

Category	Severity	Location	Status
Design Issue	● Informational	src/SeraLib.sol (04/09-905e7f7): 89~99	● Resolved

Description

In `SeraLib.sol`, each partial fill computes `executionValue0` and `executionValue1` using `Math.Rounding.Ceil`, applying independent upward rounding at the fragment level. Order progress is tracked solely through cumulative `filledAmount` of `fromAmount`, without enforcing any cross-fill reconciliation on the aggregated `toToken` output.

In fragmented execution scenarios, this can result in differences between the total `toToken` produced across multiple partial fills and the value implied by a single full execution under the same price ratio.

```
function _executionValues(MatchData calldata _match, uint256 effectiveAmount0,
uint256 effectiveAmount1) internal pure returns (uint256 executionValue0, uint256
executionValue1) {
    // amount upper bounds are already validated in `_validateOrderCommon`
    before this is invoked

    // executionValue0 = Amount of "toToken" (order1.fromToken) that order0
    expects for the given effectiveAmount0
    executionValue0 = Math.mulDiv(effectiveAmount0, _match.order0.toAmount,
_match.order0.fromAmount, Math.Rounding.Ceil);

    // executionValue1 = Amount of "toToken" (order0.fromToken) that order1
    expects for the given effectiveAmount1
    executionValue1 = Math.mulDiv(effectiveAmount1, _match.order1.toAmount,
_match.order1.fromAmount, Math.Rounding.Ceil);

    if (effectiveAmount1 < executionValue0 || effectiveAmount0 <
executionValue1) revert InvalidCostAmount();
}
```

Root Cause

This behavior arises from the combination of the following mechanics:

- Each partial fill applies independent ceiling rounding when computing proportional execution values, introducing per-fragment rounding effects;
- The system tracks completion only via cumulative `fromAmount`, without aggregating or reconciling total `toToken` across multiple fills.

As a result, for coarse-ratio orders (e.g. 11:1), a single full fill would imply a certain total `toToken`, while multiple smaller fills

may each independently round up, leading to different aggregated outcomes depending on how the order is sliced.

I Dependency on Matching Strategy

The effect of this behavior is closely tied to the execution granularity of the matching process:

- When orders are executed in fewer, larger matches, the impact of per-fragment rounding is limited.
- When execution is split into many small partial fills, rounding effects accumulate across fragments.
- In more fragmented execution patterns, the divergence between aggregated and full-fill outcomes becomes more observable.

It is also worth noting that fragmentation is not fully user-controlled:

- Match splitting is determined by the `EXECUTOR_ROLE` or system-level matching strategy;
- Orders remain subject to signature constraints and size bounds;
- Additional constraints may apply at the SOR or limit order level.

I Recommendation

We would like to confirm whether this behavior is expected to interact with the intended matching strategy, specifically the following:

- Whether per-fragment ceil rounding is an intended and accepted behavior in the pricing model
- Whether the current matching strategy is expected to support highly fragmented (small-size) executions without affecting pricing consistency

I Alleviation

[Sera, 04/13/2026]:

This is intentional. Using `Math.Rounding.Ceil` ensures users always receive at least their expected minimum output, protecting them from rounding losses. The accumulated rounding difference across partial fills is negligible (typically 1-2 wei per fill) and is absorbed by the counterparty who agreed to the price ratio. This is a standard practice in DeFi protocols to favor user protection over perfect mathematical symmetry.

OPTIMIZATIONS | SERA FX ONCHAIN

ID	Title	Category	Severity	Status
SFO-02	State Variables That Could Be Declared Immutable	Coding Issue	Optimization	● Resolved

SFO-02 | State Variables That Could Be Declared Immutable

Category	Severity	Location	Status
Coding Issue	● Optimization	src/SeraAdmin.sol (03/23-738f0a8): 30	● Resolved

Description

State variables that are not updated following deployment should be declared immutable to save gas.

```
Vault public vault;
```

Recommendation

Add the immutable attribute to state variables that never change or are set only in the constructor.

Alleviation

[Sera, 04/09/2026]:

Issue acknowledged. Changes have been reflected in the commit hash: [905e7f7ba3a669fa0b2e393afeb289e5189c04f6](#).

APPENDIX | SERA FX ONCHAIN

Audit Scope

sera-cx/orderbook-contract-v2

 src/Sera.sol

 src/Vault.sol

 src/Sera.sol

 src/SeraAdmin.sol

 src/SeraSOR.sol

 src/SeraSOR.sol

 src/Sera.sol

 src/SeraLib.sol

 src/SeraBase.sol

 src/SeraBatcher.sol

 src/SeraLib.sol

 src/SeraAdmin.sol

 src/SeraBase.sol

 src/SeraBatcher.sol

 src/SeraLib.sol

 src/Vault.sol

 src/SeraAdmin.sol

 src/SeraBase.sol

 src/SeraBatcher.sol

sera-cx/orderbook-contract-v2

 src/SeraSOR.sol src/Vault.sol src/Sera.sol src/SeraAdmin.sol src/SeraBase.sol src/SeraBatcher.sol src/SeraLib.sol src/SeraSOR.sol src/Vault.sol

Finding Categories

Categories	Description
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.
Design Issue	Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories.
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.
Coding Issue	Coding Issue findings are about general code quality including, but not limited to, coding mistakes, compile errors, and performance issues.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevate Your Web3 Journey

CertiK is the largest Web3 security platform combining formal verification with audits and comprehensive security solutions.

Sera FX Onchain Security Assessment | CertiK Assessed on Apr 30th, 2026 | Copyright © CertiK

